Universidad Tecnológica Nacional – Facultad Regional Córdoba

Cátedra de Ingeniería y Calidad de Software – 2026

Prof. Ing. Judith Meles / Ing. Laura Covaro

INFORME – TRABAJO DE INVESTIGACIÓN GRUPAL 1

Plan de despliegue: Microservicios de alto tráfico

Escenario 5 – Despliegue de Software

Comisión: 4k2

Grupo: G9

Integrantes:

- 83573 - Begliardo, Francisco Hugo
- 93686 - Brollo, Lucas Emanuel
- 85817 - Calvi, Francisco
- 85860 - Canaan, Abigail Sara
- 90023 - Carreras, Nicolás
- 85922 - Marrali, Matias
- 87440 - Mazzucco, Giuliano
- 63903 - Rodríguez, Fernando Marcos
- 86424 - Dagum , Braian David
- 86199 - Ladux, Carlos Agustin
- 90891 - Mansilla, Benjamin Felipe
- 94393 - Garro Vazquez, Matias Nicolas

Fecha de exposición: 06/11/2026

1. Introducción

¿Cómo se actualiza un avión en pleno vuelo? Esa es, en esencia, la situación de una plataforma de streaming: millones de personas la usan a toda hora y, aun así, sus equipos publican cambios todos los días.

Este informe presenta un plan de despliegue para una plataforma compuesta por decenas de microservicios interdependientes. Se describen las estrategias aplicables, los roles, los procesos y las relaciones con SCM, DevOps y las prácticas continuas, junto con la seguridad y un Plan de Release completo.

2. El escenario

Una empresa de streaming maneja su plataforma con decenas de microservicios (catálogo, recomendaciones, pagos, historial de usuarios, etc.). Muchos servicios dependen de otros y cualquier incompatibilidad puede romper funcionalidades críticas. Los desafíos que el plan debe resolver son:

- **Interdependencia:** un cambio incompatible en un servicio se propaga a los que lo consumen.
- **Cero downtime:** millones de usuarios están conectados 24/7.
- **Cambios graduales:** se debe preservar la compatibilidad de los contratos de API entre versiones.

3. Despliegue según el tipo de producto

Es una plataforma distribuida en la nube, empaquetada en contenedores y ejecutada sobre un orquestador (Kubernetes). A diferencia de un producto instalable o una app móvil, la empresa controla el entorno de producción, por lo que puede desplegar con alta frecuencia, de forma independiente por servicio y revertir en minutos.

4. Estrategias de despliegue

No existe una estrategia única: cada una equilibra de manera distinta velocidad, riesgo y costo. A continuación se describen las seis consideradas, con su aplicación y las herramientas involucradas.

4.1 Rolling update

Reemplaza progresivamente las instancias de la versión anterior por las de la nueva, en lotes pequeños, y verifica que cada instancia esté sana antes de continuar. Siempre hay capacidad disponible, por lo que no hay downtime. Durante la transición conviven dos versiones, por lo que exige compatibilidad hacia atrás.

En este escenario: se usa para servicios de bajo riesgo y como base de las demás estrategias.

Herramientas: Kubernetes (Deployment), Helm.

4.2 Canary release

Dirige una fracción pequeña del tráfico real (por ejemplo 1 % → 5 % → 25 % → 100 %) a la nueva versión y analiza métricas en cada paso. Si los indicadores se degradan, el tráfico vuelve a la versión estable. Limita el impacto de un error a una minoría de usuarios.

En este escenario: es la estrategia principal para servicios de alto tráfico, como catálogo.

Herramientas: Argo Rollouts, Flagger, Istio / Linkerd, Prometheus.

4.3 Blue/Green

Mantiene dos entornos idénticos: el activo (blue) y el nuevo (green). Tras validar el green, se cambia todo el tráfico de una sola vez; el rollback consiste en volver al blue. Ofrece un cambio y una reversión instantáneos, a costa de duplicar temporalmente la infraestructura.

En este escenario: se reserva para servicios críticos, como pagos, donde un error tiene impacto económico directo.

Herramientas: Kubernetes Services / Ingress, Argo Rollouts, API Gateway.

4.4 Feature flags

Separan desplegar de liberar: el código llega a producción con la funcionalidad apagada y se activa luego, de forma gradual y reversible, sin necesidad de un nuevo despliegue. Permiten apagar de inmediato una función problemática.

En este escenario: el Product Owner decide cuándo activar una función según el valor de negocio y el estado de las métricas.

Herramientas: LaunchDarkly, Unleash, OpenFeature.

4.5 Dark launch / shadow traffic

La nueva versión recibe una copia del tráfico de producción, pero sus respuestas no llegan al usuario. Permite comparar su comportamiento y rendimiento con la versión actual bajo carga real y sin riesgo.

En este escenario: se aplica a servicios como recomendaciones, para validar un nuevo algoritmo antes de exponerlo.

Herramientas: Istio (mirroring), API Gateway, proxies.

4.6 Expand / Contract (Parallel Change)

Los cambios incompatibles en una API o en una base de datos se hacen en tres fases: expandir (agregar lo nuevo conviviendo con lo viejo), migrar a los consumidores y contraer (retirar lo viejo). Garantiza la compatibilidad de contratos y permite revertir en cualquier punto.

En este escenario: es la técnica que asegura que los cambios se apliquen gradualmente sin romper a los servicios dependientes.

Herramientas: Versionado de APIs, Pact (contract testing), Flyway / Liquibase.

4.7 Herramientas transversales

Docker (empaquetado), registry de imágenes, Git, Argo CD (GitOps), Terraform (infraestructura como código) y Prometheus, Grafana y OpenTelemetry (observabilidad).

5. Roles involucrados

Rol	Responsabilidad en el despliegue
Desarrolladores (dueños del servicio)	Construyen, prueban y despliegan su servicio; responden por él en producción («you build it, you run it»).
DevOps / SRE	Mantienen pipelines, observabilidad, SLOs y respuesta a incidentes.
Equipo de plataforma	Provee el clúster, el service mesh y las plantillas de pipeline como producto interno.

Rol	Responsabilidad en el despliegue
QA	Define y automatiza pruebas de contrato, regresión y smoke tests.
Release Manager / líder técnico	Coordina ventanas, aprueba cambios de alto riesgo y comunica.
Product Owner	Decide cuándo activar funcionalidades (feature flags) según valor de negocio.
Seguridad (DevSecOps)	Define políticas, escaneos y gestión de secretos.

6. Proceso de despliegue

Flujo propuesto, de commit a producción:

Commit → CI (build + tests unitarios) → imagen Docker versionada → escaneo de seguridad → contract tests → despliegue a staging → canary en producción → análisis de métricas → promoción al 100 % o rollback automático.

- Cada servicio tiene su propio pipeline, lo que permite desplegarlo de forma independiente.
- La promoción entre etapas es automática si se cumplen los criterios de calidad (quality gates).
- Los cambios de alto riesgo requieren aprobación manual del líder técnico o Release Manager.

7. Relación con SCM (Gestión de la Configuración del Software)

- Control de versiones en Git con trunk-based development y ramas de vida corta.
- Versionado semántico (MAJOR.MINOR.PATCH) de servicios y APIs: el cambio MAJOR indica incompatibilidad.
- Artefactos inmutables: una imagen con un tag dado nunca se modifica y se almacena en un registry.
- Infraestructura y configuración como código, versionadas junto al servicio (GitOps: el repositorio es la fuente de verdad).
- Trazabilidad: cada versión desplegada se vincula a un commit, un build y un cambio, lo que permite saber a qué versión volver.

8. Relación con DevOps

- Cultura de responsabilidad compartida entre desarrollo y operaciones.
- Automatización de punta a punta (build, prueba, despliegue, infraestructura).
- Observabilidad compartida: logs, métricas y trazas distribuidas accesibles para todos.
- Platform engineering: la plataforma interna reduce la carga cognitiva de los equipos.
- Postmortems sin culpa y mejora continua.

9. Relación con las prácticas continuas

- **Integración continua (CI):** cada commit se integra, compila y prueba automáticamente.
- **Entrega continua (CD):** todo cambio que pasa el pipeline queda listo para producción; es la práctica que habilita despliegues frecuentes y seguros.

- **Despliegue continuo:** en servicios de bajo riesgo, la promoción a producción es totalmente automática.
- **Testing continuo:** pruebas unitarias, de contrato, de integración y smoke tests en cada etapa.
- **Monitoreo continuo:** las métricas en producción realimentan la decisión de avanzar o revertir.

10. Seguridad en el despliegue

- Escaneo de imágenes y dependencias (por ejemplo Trivy) y generación de SBOM.
- Firma y verificación de artefactos (por ejemplo cosign); el clúster solo acepta imágenes firmadas.
- Gestión de secretos en un gestor dedicado (por ejemplo Vault), nunca en el código ni en las imágenes.
- mTLS entre servicios mediante el service mesh.
- Network policies y RBAC con principio de mínimo privilegio.
- SAST y DAST integrados en el pipeline; registro de auditoría de quién desplegó qué y cuándo.

11. Plan de Release

Se toma como ejemplo el despliegue de la versión 2.4.1 del microservicio de catálogo.

Ítem	Definición
Entorno de destino	Producción multi-región, precedida por staging.
Tipo de despliegue	Progresivo (canary) sobre un servicio en ejecución, sin interrupción.
Tipo de infraestructura	Clústeres Kubernetes en la nube con autoscaling, service mesh y API Gateway.
Artefacto y versión de retorno	Imagen Docker catalogo:2.4.1 (firmada, con digest). Versión anterior a la que volver: catalogo:2.4.0, disponible en el registry.
Cómo se instala	El pipeline CD actualiza el manifiesto (Helm/GitOps) en el repositorio; Argo CD sincroniza el clúster y Argo Rollouts gestiona el canary.
Dependencias	Servicios consumidores (recomendaciones, búsqueda), base de datos del catálogo, cola de mensajes. Los contratos con ellos están verificados.
Prerrequisitos	Contract tests en verde; migraciones de base de datos compatibles hacia atrás; feature flags configurados; dashboards y alertas listos; versión anterior disponible.
Horario y justificación	Franja de bajo tráfico de cada región, evitando estrenos y picos. Con canary también es viable en horario laboral, para contar con el equipo ante un problema. Se evitan viernes y vísperas de eventos.
Forma de despliegue (técnica)	Contenedores sobre Kubernetes; actualización declarativa vía GitOps.
Estrategia para reducir riesgos	Canary 1 % → 5 % → 25 % → 100 % con análisis automático de métricas en cada paso; feature flags para funcionalidades nuevas; expand/contract para cambios de API o base de datos.
Monitoreo	Latencia (p95/p99), tasa de errores, saturación de recursos, métricas de negocio (reproducciones iniciadas), trazas distribuidas y alertas por SLO.

Ítem	Definición
Pruebas post-despliegue	Health checks y readiness probes, smoke tests de los flujos críticos (listar catálogo, abrir un título), verificación de contratos con consumidores.
Comunicación y aprobaciones	Antes: aviso a soporte, producto y equipos dependientes; aprobación del líder técnico o Release Manager. Después: reporte del resultado en el canal de releases.
Plan de rollback / contingencia	Automático si se superan umbrales de error o latencia: el tráfico vuelve a 2.4.0 en minutos. Si hubo migración de datos, la fase de contracción se aplica solo cuando la versión nueva está estable. Si el rollback falla: apagar el feature flag, escalar a on-call, activar el proceso de incidentes y realizar un postmortem.

12. Aporte original: matriz de decisión de estrategia

Proponemos no usar una única estrategia para todos los servicios, sino elegirla según el riesgo y las características de cada microservicio.

Tipo de microservicio	Estrategia sugerida	Justificación
Pagos / facturación	Blue/Green + aprobación manual	Un error tiene impacto económico directo; se necesita cambio y vuelta atrás instantáneos.
Recomendaciones	Canary + feature flag + shadow traffic	Se puede comparar el nuevo algoritmo con el anterior sin afectar al usuario.
Catálogo	Canary automatizado	Alto tráfico y lectura intensiva; las métricas detectan rápido las regresiones.
Historial de usuarios	Rolling update + expand/contract	Hay cambios de esquema de datos que exigen compatibilidad entre versiones.
Servicios internos de bajo riesgo	Rolling update con despliegue continuo	El riesgo es bajo; se prioriza la velocidad.

13. Conclusiones

En una plataforma de microservicios de alto tráfico no se despliega todo de una vez: se despliega poco, de a poco y con métricas que deciden si se avanza o se vuelve atrás. La combinación de contenedores, GitOps, canary, feature flags y contract testing permite liberar cambios con frecuencia sin interrumpir el servicio ni romper los contratos entre servicios.

El aporte del grupo es elegir la estrategia según el riesgo de cada servicio, en lugar de aplicar la misma a todos. El mejor despliegue es el que los usuarios no notan.

14. Referencias

- Stackpole, B. & Hanrion, P. (2019). Software Deployment, Updating, and Patching. CRC Press.
- Blokdyk, G. (2021). Software Deployment A Complete Guide – 2019 Edition. 5STARCooks.
- Humble, J. & Farley, D. (2010). Continuous Delivery. Addison-Wesley.
- Newman, S. (2021). Building Microservices (2.ª ed.). O'Reilly.
- Beyer, B. et al. (2016). Site Reliability Engineering. Google / O'Reilly.

- Fowler, M. Artículos «BlueGreenDeployment», «CanaryRelease» y «ParallelChange». martinfowler.com.
- Documentación oficial de Kubernetes (kubernetes.io), Argo Rollouts y Argo CD.
- Claude (Anthropic). Asistente de inteligencia artificial utilizado como apoyo en la estructuración y redacción de este informe (2026).

15. Agradecimientos

Agradecemos a la cátedra de Ingeniería y Calidad de Software, a las profesoras Judith Meles y Laura Covaro, y a nuestros compañeros de comisión por su atención y sus aportes.